Task 2: System Design

Agentic AI for Healthcare — Sleep Apnea Detection

Prepared by: Sirigiri Venkateswara Adithya | B.Tech CSE (Data Science & Analytics) | IIIT Sonepat | CPI 9.43

Submitted to: Dr. K Naveen Kumar

2.1 Healthcare Use Case

The proposed Agentic AI system targets automated Sleep Apnea Severity Screening — one of the most clinically impactful yet underserved problems in digital health. Sleep apnea affects an estimated 936 million adults globally, and over 80% of cases remain undiagnosed due to the high cost and limited accessibility of Polysomnography (PSG), the clinical gold standard.

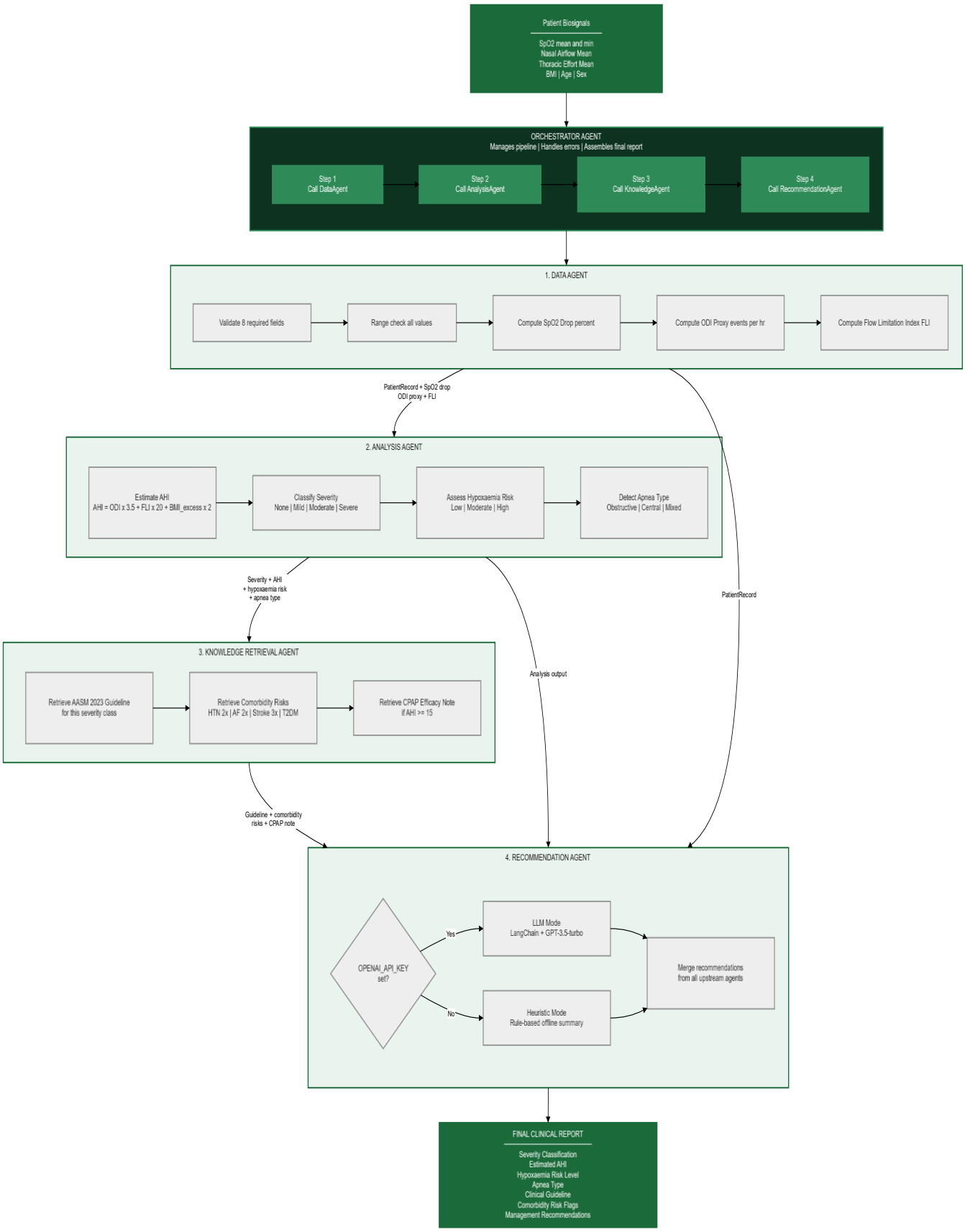
The system ingests home-based physiological measurements — SpO2 (mean and minimum), nasal airflow, thoracic respiratory effort, BMI, age, and sex — and produces a complete clinical report without physician involvement at the inference stage. The report includes severity classification (None / Mild / Moderate / Severe), Apnea-Hypopnea Index (AHI) estimation, hypoxaemia risk assessment, apnea type identification (Obstructive / Central / Mixed), evidence-based management guidelines aligned with AASM 2023 standards, and comorbidity risk flags.

2.2 Overall Architecture

The system follows a sequential multi-agent pipeline coordinated by a central OrchestratorAgent. Each agent has a single, well-defined responsibility and communicates through a standardised AgentOutput dataclass containing a status flag, a structured data payload, a reasoning trace, and a list of recommendations. The output of each agent becomes the input of the next, creating a traceable, auditable clinical reasoning chain.

The architecture diagram below illustrates the complete pipeline — from raw patient biosignals through all five agents to the final clinical report:

Figure 2.1: Multi-Agent Pipeline Architecture


Patient Biosignals
SpO2 mean and min
Nasal Airflow Mean
Thoracic Effort Mean
BMI | Age | Sex
ORCHESTRATOR AGENT
Manages pipeline | Handles errors | Assembles final report
Step 1
Call DataAgent
Step 2
Call AnalysisAgent
Step 3
Call KnowledgeAgent
Step 4
Call RecommendationAgent
1. DATA AGENT
Validate 8 required fields
Range check all values
Compute SpO2 Drop percent
Compute ODI Proxy events per hr
Compute Flow Limitation Index FLI
PatientRecord + SpO2 drop
ODI proxy + FLI
2. ANALYSIS AGENT
Estimate AHI
AHI = ODI x 3.5 + FLI x 20 + BMI_excess x 2
Classify Severity
None | Mild | Moderate | Severe
Assess Hypoxaemia Risk
Low | Moderate | High
Detect Apnea Type
Obstructive | Central | Mixed
Severity + AHI
+ hypoxaemia risk
+ apnea type
PatientRecord
Analysis output
3. KNOWLEDGE RETRIEVAL AGENT
Retrieve AASM 2023 Guideline
for this severity class
Retrieve Comorbidity Risks
HTN 2x | AF 2x | Stroke 3x | T2DM
Retrieve CPAP Efficacy Note
if AHI >= 15
Guideline + comorbidity
risks + CPAP note
4. RECOMMENDATION AGENT
OPENAI_API_KEY
set?
Yes
No
LLM Mode
LangChain + GPT-3.5-turbo
Heuristic Mode
Rule-based offline summary
Merge recommendations
from all upstream agents
FINAL CLINICAL REPORT
Severity Classification
Estimated AHI
Hypoxaemia Risk Level
Apnea Type
Clinical Guideline
Comorbidity Risk Flags
Management Recommendations

2.3 Agent Roles and Responsibilities

The system comprises five agents. The first four are specialist agents each responsible for one stage of the clinical reasoning process. The fifth — the OrchestratorAgent — acts as the pipeline manager, instantiating and coordinating all specialist agents.

Table 2.1: Agent Summary

Agent	Role	Input	Output
DataAgent	Validation & feature engineering	Raw patient dict	Validated PatientRecord + derived features (SpO2 drop, ODI proxy, FLI)
AnalysisAgent	AHI estimation & severity classification	PatientRecord + derived features	AHI, severity class, hypoxaemia risk, apnea type
KnowledgeRetrievalAgent	Clinical guideline retrieval	Severity + AHI	AASM guidelines, comorbidity risks, CPAP evidence
RecommendationAgent	Clinical summary synthesis	All upstream AgentOutputs	Clinical summary + management plan
OrchestratorAgent	Pipeline coordination	Raw patient dict	Final structured clinical report (JSON)

2.3.1 DataAgent

The DataAgent is the entry point of the pipeline. It accepts raw patient data as a Python dictionary and performs three sequential operations before passing a clean, enriched record to downstream agents.

- **Field presence validation —** Checks that all 8 required fields exist. Returns an error AgentOutput immediately if any field is missing, halting the pipeline.
- **Range validation —** Verifies that each physiological value falls within a medically sensible range (e.g., SpO2: 70-100%, BMI: 10-80, airflow: 0-1). Out-of-range values trigger warnings but do not halt processing.
- **Feature engineering —** Computes three derived clinical features: SpO2 Drop % (mean minus minimum SpO2), ODI Proxy (SpO2_drop / 3.0, estimating desaturation events per hour), and Flow-Limitation Index or FLI ((1 - nasal_airflow) x thoracic_effort). A high FLI

above 0.35 with reduced airflow but elevated thoracic effort is the hallmark signature of obstructive apnea.

2.3.2 AnalysisAgent

The AnalysisAgent receives the enriched PatientRecord from the DataAgent and performs the core clinical reasoning. It uses a heuristic formula to estimate the Apnea-Hypopnea Index when no measured value is available:

```
AHI = ODI_proxy × 3.5 + FLI × 20 + BMI_excess × 2
```

The agent then classifies severity using AASM 2023 thresholds (None: AHI < 5, Mild: 5-15, Moderate: 15-30, Severe: > 30 events/hr), assesses hypoxaemia risk based on minimum SpO2 (Low above 90%, Moderate between 85-90%, High below 85%), and identifies the likely apnea type using FLI thresholds.

2.3.3 KnowledgeRetrievalAgent

The KnowledgeRetrievalAgent queries a structured knowledge base at inference time, simulating the behaviour of a production RAG (Retrieval-Augmented Generation) system. Based on the severity class and AHI received from the AnalysisAgent, it retrieves the relevant AASM 2023 management guideline, applicable comorbidity risk data (hypertension 2x risk, atrial fibrillation 2x risk, stroke 3x risk for severe cases, type-2 diabetes), and CPAP efficacy evidence. In a production deployment, this agent would query a FAISS or Chroma vector store loaded with actual clinical PDF guidelines.

2.3.4 RecommendationAgent

The RecommendationAgent synthesises the outputs of all three upstream agents into a structured clinical recommendation. It operates in two modes depending on the environment:

- **Heuristic Mode (default, offline) —** Deterministic rule-based summary assembled from structured data. Works with zero external dependencies.
- **LLM Mode (when OPENAI_API_KEY is set) —** Sends structured JSON context to GPT-3.5-turbo via LangChain with a clinical AI assistant system prompt, generating a richer natural-language clinical summary. Automatically falls back to heuristic mode if the API call fails.

2.3.5 OrchestratorAgent

The OrchestratorAgent instantiates all four specialist agents and manages the sequential pipeline execution. After each stage it checks the agent's status — if any agent returns an error, the pipeline halts gracefully and returns a partial report with the error details rather than crashing. It assembles the final structured report containing the complete patient assessment, per-agent reasoning traces, clinical summary, severity, estimated AHI, and final merged recommendations.

2.4 Data Flow

The diagram below shows how data moves between agents at each stage of the pipeline. Every inter-agent communication follows the standardised AgentOutput interface, making the pipeline fully modular — any agent can be replaced or upgraded without changing the others.

Table 2.2: Inter-Agent Data Flow

Stage	From → To	What is Passed
Stage 1	Raw Input → DataAgent	Python dict with 8 patient fields
Stage 2	DataAgent → AnalysisAgent	PatientRecord dataclass + SpO2 drop, ODI proxy, FLI
Stage 3	AnalysisAgent → KnowledgeAgent	Severity class + AHI value + hypoxaemia risk
Stage 4	All three → RecommendationAgent	DataOutput + AnalysisOutput + KnowledgeOutput
Stage 5	All four → OrchestratorAgent	All AgentOutputs → assembled into final JSON report

2.5 Technology Stack

Table 2.3: Technology Stack

Component	Technology
Agent orchestration	Python 3.10+ — custom OrchestratorAgent class
LLM integration	LangChain + OpenAI GPT-3.5-turbo (optional)
Data processing	Pandas, NumPy
Evaluation	scikit-learn — accuracy, F1, confusion matrix
Knowledge base (prototype)	In-memory Python dictionary — AASM 2023 guidelines
Knowledge base (production)	FAISS / Chroma vector store + RAG pipeline

Component	Technology
Dataset	Synthetic PSG data — 100 patients, 4 severity classes
Interface	Jupyter Notebook (main.ipynb)

2.6 Key Design Decisions

Offline-first design: The system runs 100% without an internet connection or API key. LLM enhancement is additive, not required. This ensures the prototype is demonstrable in any environment.

Standardised AgentOutput interface: Every agent returns the same AgentOutput dataclass structure. This standardisation makes the pipeline modular — any agent can be replaced as long as it respects the interface contract.

Graceful error propagation: If any agent returns status='error', all downstream agents short-circuit gracefully rather than crashing. The OrchestratorAgent assembles a partial report with error details.

Dual-mode RecommendationAgent: Automatic LLM/heuristic switching ensures the system degrades gracefully and is immediately upgradeable to LLM-powered output by simply setting an environment variable.

Production upgrade path: The KnowledgeRetrievalAgent is designed with a clear upgrade path — the in-memory dictionary can be replaced with a FAISS vector store and the AnalysisAgent heuristic can be replaced with a trained 1D-CNN — both without modifying any other component in the pipeline.